MINI ASSESSMENT - LAB REPORT

Name:	S Thaniya	**Department:**	GEN-AI
Register No:	99230040774	**Date:**	11-09-2026

1. AIM

To design and develop an interactive, rule-based conversational chatbot in Python using conditional if-elif-else branch statements and an interactive execution loop to process user prompts and return contextually appropriate responses.

2. PROCEDURE

1. Initialize a continuous execution loop using while True.

2. Ask the user for their name and use the name in chatbot responses.

3. Read user input using input().

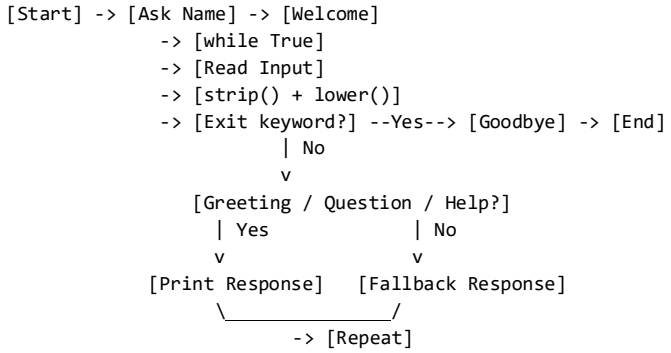
4. Standardize input using .strip() and .lower().

5. Check bye, exit, and quit and terminate using break.

6. Use if-elif branches for greetings, questions, and help.

7. Use else as a fallback for unknown messages.

3. CHATBOT FLOW



4. PYTHON CHATBOT IMPLEMENTATION

```
def rule_based_bot():

    name = input("What is your name? ")

    print(f"\nBot: Hello {name}! I am your Python chatbot.")
    print("Bot: Type 'bye', 'exit', or 'quit' to end the conversation.\n")

    while True:

        user_input = input("You: ").strip().lower()

        if user_input in ["bye", "exit", "quit"]:
            print(f"Bot: Goodbye {name}! Have a great day!")
            break

        elif "hello" in user_input or "hi" in user_input or "hey" in user_input:
            print(f"Bot: Hello {name}! How can I help you today?")

        elif "how are you" in user_input:
            print("Bot: I am doing great, thank you! How about you?")

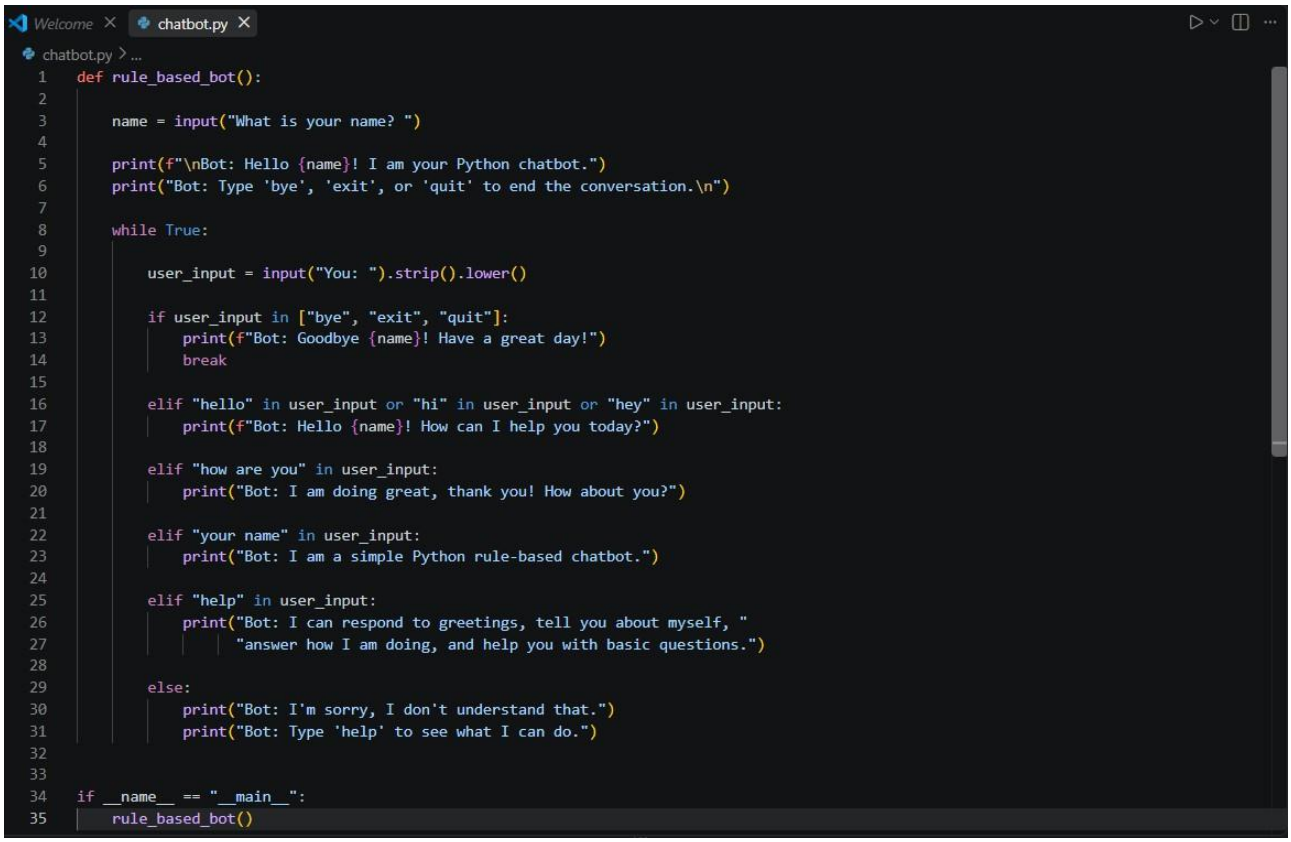
        elif "your name" in user_input:
            print("Bot: I am a simple Python rule-based chatbot.")

        elif "help" in user_input:
            print("Bot: I can respond to greetings, tell you about myself,")
            print("Bot: answer how I am doing, and help you with basic questions.")

        else:
            print("Bot: I'm sorry, I don't understand that.")
            print("Bot: Type 'help' to see what I can do.")


if _name_== "_main_":
    rule_based_bot()
```

Figure 1. Python chatbot implementation in VS Code


Welcome
chatbot.py
chatbot.py > ...
def rule_based_bot():
name = input("What is your name? ")
print(f"\nBot: Hello {name}! I am your Python chatbot.")
print("Bot: Type 'bye', 'exit', or 'quit' to end the conversation.\n")
while True:
user_input = input("You: ").strip().lower()
if user_input in ["bye", "exit", "quit"]:
print(f"Bot: Goodbye {name}! Have a great day!")
break
elif "hello" in user_input or "hi" in user_input or "hey" in user_input:
print(f"Bot: Hello {name}! How can I help you today?")
elif "how are you" in user_input:
print("Bot: I am doing great, thank you! How about you?")
elif "your name" in user_input:
print("Bot: I am a simple Python rule-based chatbot.")
elif "help" in user_input:
print("Bot: I can respond to greetings, tell you about myself, "
"answer how I am doing, and help you with basic questions.")
else:
print("Bot: I'm sorry, I don't understand that.")
print("Bot: Type 'help' to see what I can do.")
if __name__ == "__main__":
rule_based_bot()

5. CHATBOT EXECUTION AND TEST OUTPUT

The chatbot was executed successfully in the VS Code terminal. The test demonstrates name input, greeting recognition, question handling, help response, unknown-input handling, and graceful termination.

User Input	Expected Response	Status
hello / HELLO / hi / hey hello	Greeting	PASS
how are you	Status response	PASS
what is your name	Identity response	PASS
help	Help response	PASS
can you dance	Fallback response	PASS
bye	Goodbye and exit	PASS

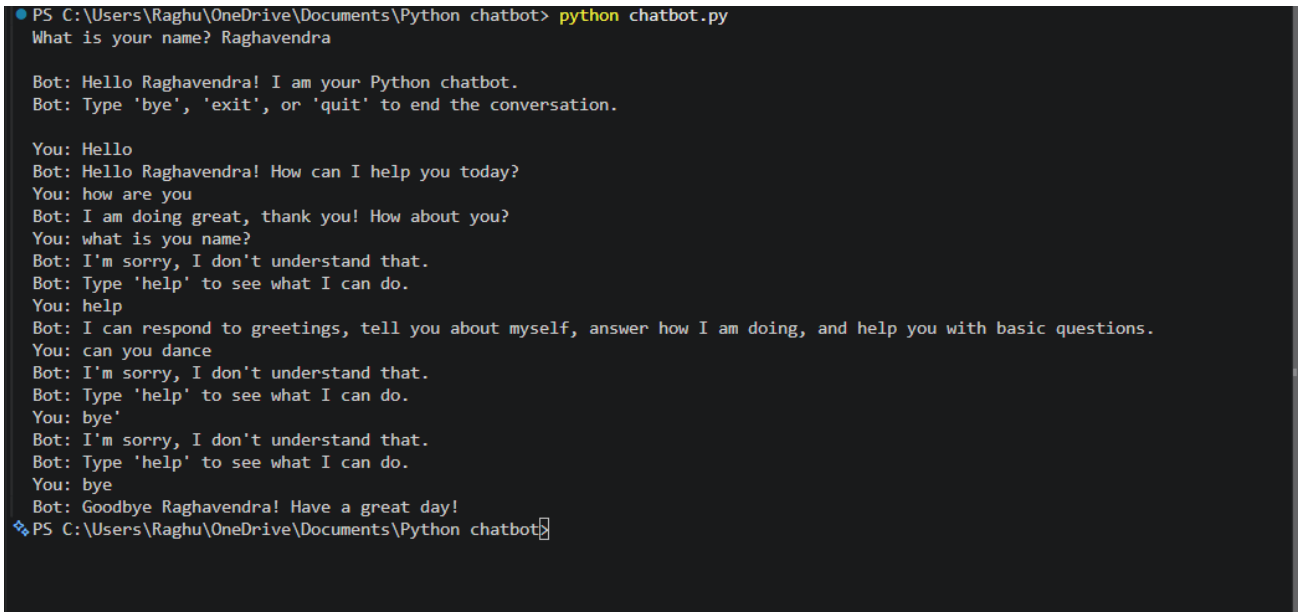


Figure 2. Terminal execution and test output

6. RESULT

The rule-based Python chatbot was successfully implemented and executed. It accepts user messages continuously, preprocesses input, recognizes greetings and questions, handles unknown messages, and exits when the user enters bye, exit, or quit.

7. CONCLUSION

The mini assessment successfully demonstrates a simple conversational system using Python if-elif-else branching and an interactive while loop. The chatbot follows deterministic keyword-based rules and provides appropriate responses.